Jala University

Desarrollo Web

Proyecto Final

Construye una Aplicación de Página Única (SPA) rápida y responsiva con frameworks modernos, ofreciendo experiencias de usuario fluidas.

Este material es sólo para uso académico.
Está prohibido distribuirlo o publicarlo sin autorización previa por escrito.
2026 Universidad Jala | Todos los derechos están reservados.

Proyecto Final

Este proyecto servirá como una demostración para implementar principios modernos de desarrollo frontend, incluyendo diseño responsivo, gestión de estado, reactividad, integración de API, accesibilidad y optimización del rendimiento.

Visión General del Proyecto

Construir una Aplicación de Página Única (SPA) para gestionar y explorar libros en una biblioteca universitaria. Los usuarios podrán navegar por un catálogo de libros obtenido de una API pública, ver información detallada sobre cada libro e interactuar con funcionalidades como búsqueda por palabras clave, filtrado por categorías y opciones de ordenación por autor, género o popularidad. Además, los usuarios pueden tomar prestados libros, hacer seguimiento de los elementos prestados y ver fechas de vencimiento.

Requisitos Centrales del Sistema

Requisito	Descripción
1. Navegación de Libros	• Mostrar libros obtenidos de una API pública (ej., API de Open Library o Google Books API) • Mostrar detalles del libro, incluyendo imagen de portada, título, autor, descripción, género y estado de disponibilidad.
2. Búsqueda y Filtrado	• Permitir a los usuarios buscar libros por título, autor o palabras clave. • Implementar filtrado basado en categorías (ej., ficción, ciencia, historia) y ordenación por fecha de publicación, autor o popularidad. • Asegurar manejo eficiente de datos para conjuntos de datos grandes.

3. Sistema de Préstamos	• Permitir a los usuarios tomar prestados libros y hacer seguimiento de fechas de vencimiento. • Implementar un sistema de reservas para libros actualmente prestados. • Almacenar historial de préstamos del usuario de forma persistente (almacenamiento local o base de datos).
4. Página de Detalles del Libro	• Proporcionar una página dedicada para cada libro con información detallada. • Incluir un rastreador de estado de lectura (ej., “Leyendo”, “Completado”, “Lista de deseos”).
5. Lista de Deseos (Función Avanzada Opcional)	• Permitir a los usuarios crear y gestionar una lista de deseos de libros que quieren leer en el futuro.

	• Implementar notificaciones para cuando los libros en la lista de deseos estén disponibles para préstamo.
6. Diseño Responsivo	• Asegurar una experiencia de usuario fluida en todos los dispositivos. • Usar un enfoque mobile-first para los diseños.
7. Interfaz de Usuario y Accesibilidad	• Implementar navegación por teclado y roles ARIA. • Diseñar una interfaz visualmente atractiva e intuitiva.
8. Optimización del Rendimiento	• Implementar carga perezosa para imágenes y otros recursos. • Optimizar la aplicación para tiempos de carga más rápidos y mejor rendimiento.

9. Medidas de Seguridad	• Implementar validación y sanitización de entradas para prevenir ataques XSS (Cross-Site Scripting). • Asegurar transmisión segura de datos usando HTTPS. • Proporcionar mecanismos seguros de autenticación de usuarios. • Añadir validación de formularios del lado del cliente para envíos seguros y libres de errores.
10. Construcción y Despliegue	• Implementar archivos de entorno para desarrollo y producción • Desplegar la aplicación web en despliegue Serverless o PaaS

Stack Tecnológico

Componente	Tecnología
Lenguaje	TypeScript
Framework Frontend	React
Gestión de Estado	React Context API o Redux Toolkit
Estilizado	CSS Modules, styled-components, o una biblioteca de componentes (ej., Material-UI, Chakra UI)
Integración de API	Fetch API o Axios
Enrutamiento	React Router DOM
Persistencia de Datos	Local Storage o un servicio backend (ej., Firebase, Supabase)
Despliegue	Vercel, Netlify, o AWS Amplify

Arquitectura



Una parte importante del proyecto final es **pensar sobre la arquitectura** de tu sistema basándose en los requisitos centrales. Algunas ideas serán proporcionadas y discutidas en las clases magistrales así como en las sesiones prácticas.

Cronograma de Desarrollo Propuesto

Semana 1: Planificación y Análisis de Requisitos

- Definir el alcance del proyecto, objetivos y entregables clave.
- Detallar todos los requisitos funcionales y no funcionales.
- Investigar y seleccionar la API pública de libros.
- Finalizar la pila tecnológica (TypeScript, React, etc.).

Semana 2: Diseño y Arquitectura

- Crear diagramas arquitectónicos de alto nivel para el frontend.
- Diseñar el modelo de datos para información de usuarios e historial de préstamos.
- Esbozar maquetas UI/UX para páginas clave (ej., búsqueda, detalles del libro).

Semana 3: Implementación (Características Principales)

- Inicializar el proyecto React con TypeScript.
- Implementar navegación y visualización de libros desde la API.

- Construir funcionalidad de búsqueda, filtrado y ordenamiento.

Semana 4: Implementación (Características de Usuario)

- Crear la página dedicada de detalles del libro.
- Implementar autenticación básica de usuarios (iniciar/cerrar sesión).
- Desarrollar el sistema de préstamos y seguimiento de fechas de vencimiento.
- Realizar una revisión y demostración a mitad del proyecto.

Semana 5: Desarrollo (Características Avanzadas)

- Implementar la característica opcional de lista de deseos.
- Agregar un rastreador de estado de lectura para libros.
- Integrar almacenamiento local para persistencia de datos.

Semana 6: Pruebas y Aseguramiento de Calidad

- Escribir y ejecutar pruebas unitarias e de integración.
- Identificar y corregir errores.
- Optimizar el rendimiento de la aplicación (ej., carga perezosa).

Semana 7: Despliegue y Mantenimiento

- Realizar una revisión final de todas las características.
- Desplegar la aplicación en una plataforma sin servidor (ej., Vercel, Netlify).
- Crear documentación del proyecto.
- Entrega final del proyecto y cierre.

¡Gracias!

Este es el final de la definición del proyecto capstone.